



廣東工業大學

数字集成电路设计课程实验

报告题目 数字集成电路设计课程实验

课程名称 数字集成电路设计课程实验

学 院 集成电路学院

专业班级 23 级集成 3 班

学 号 3123009487

姓 名 黄海鸿

2026 年 4 月 18 日

目录

一 实验 1：基于 FPGA 的按键和 LED 灯控制	1
1.1 实验目的.....	1
1.2 设计方案.....	1
1.2.1 系统功能概述	1
1.2.2 硬件资源分配	1
1.2.3 端口定义	1
1.2.4 实验设备	2
1.3 RTL 模块设计.....	2
1.4 约束脚本设计.....	2
1.5 实验结果.....	3
1.5.1 结构图	3
1.5.2 FPGA 原型验证.....	4
1.6 结论.....	4
二 实验 2：基于计数器的循环灯设计	5
2.1 实验目的.....	5
2.2 设计方案.....	5
2.2.1 系统功能概述	5
2.2.2 硬件资源分配	5
2.2.3 端口定义	5
2.2.4 实验设备	6

2.3 RTL 模块设计.....	6
2.4 约束脚本设计.....	7
2.5 仿真文件设计.....	7
2.5.1 仿真目标.....	7
2.5.2 测试向量设计.....	7
2.5.3 testbench.....	8
2.6 实验结果.....	8
2.6.1 仿真波形图.....	8
2.6.2 结构图.....	9
2.6.3 FPGA 原型验证.....	9
2.7 结论.....	10
三 实验 3：基于新思科技套件的数电设计、前仿和综合	11
3.1 实验目的.....	11
3.2 设计方案.....	11
3.2.1 系统功能概述.....	11
3.2.2 端口定义.....	11
3.2.3 实验设备.....	11
3.3 RTL 模块设计.....	12
3.4 仿真文件设计.....	12
3.4.1 仿真目标.....	12
3.4.2 测试向量设计.....	12
3.4.3 testbench.....	13

3.5 VCS 仿真结果	14
3.5.1 仿真波形	14
3.5.2 log	15
3.6 DC 综合结果	16
3.6.1 第一次综合	16
3.6.2 第二次综合	17
3.7 结论	18
四 总结	19

一 实验 1：基于 FPGA 的按键和 LED 灯控制

1.1 实验目的

- 1) 熟悉 VIVADO 软件开发环境；
- 2) 熟悉 AX7Z020 开发板资源；
- 3) 熟悉 FPGA 的设计流程；

1.2 设计方案

1.2.1 系统功能概述

设计一个基于 FPGA 的按键控制 LED 灯系统。使用 AX7Z020 开发板上的一个按键（例如 KEY1）控制四个 LED 灯（例如 LED0~LED3）。按下按键时，四个 LED 同时点亮；松开按键时，四个 LED 同时熄灭。功能简单，旨在熟悉 VIVADO 开发流程和 FPGA 板级资源。

1.2.2 硬件资源分配

- 1) **按键输入**：选择开发板上的一个独立按键 KEY1，按键按下为低电平，松开为高电平，对应的 FPGA 引脚查阅 AX7Z020 用户手册后可知为 M15。
- 2) **LED 输出**：选择四个独立 LED（LED0~LED3），每个 LED 对应一个 FPGA 引脚，引脚分配分别为 J14, K14, J18, H18。LED 引脚为共阳极，向引脚提供低电平时 LED 点亮。

1.2.3 端口定义

端口名称	方向	位宽	说明
clk	input	1	时钟(暂不使用)
rst_n	input	1	复位(暂不使用)
key	input	1	按键输入

led	output	4	LED 输出
-----	--------	---	--------

1.2.4 实验设备

- 1) 安装有 VIVADO 软件的 PC 机;
- 2) AX7Z020 开发板;

1.3 RTL 模块设计

```

1/  // Revision 0.01 - File Created
18  // Additional Comments:
19  //
20  //////////////////////////////////////
21
22
23  module test
24  (
25      input clk,
26      input rst_n,
27
28      input key,
29      output [3:0] led
30  );
31
32      assign led = {4{key}};
33
34  endmodule
35

```

由上图可得，实验使用 assign 语句实现组合逻辑，若：

按键按下(key == 0)，则 led = 4'b0000;

按键松开(key == 1)，则 led = 4'b1111;

1.4 约束脚本设计

根据 AX7Z020 原理图，查找按键和 LED 对应的 FPGA 管脚号，在 VIVADO 中创建 XDC 约束文件，如下图所示：

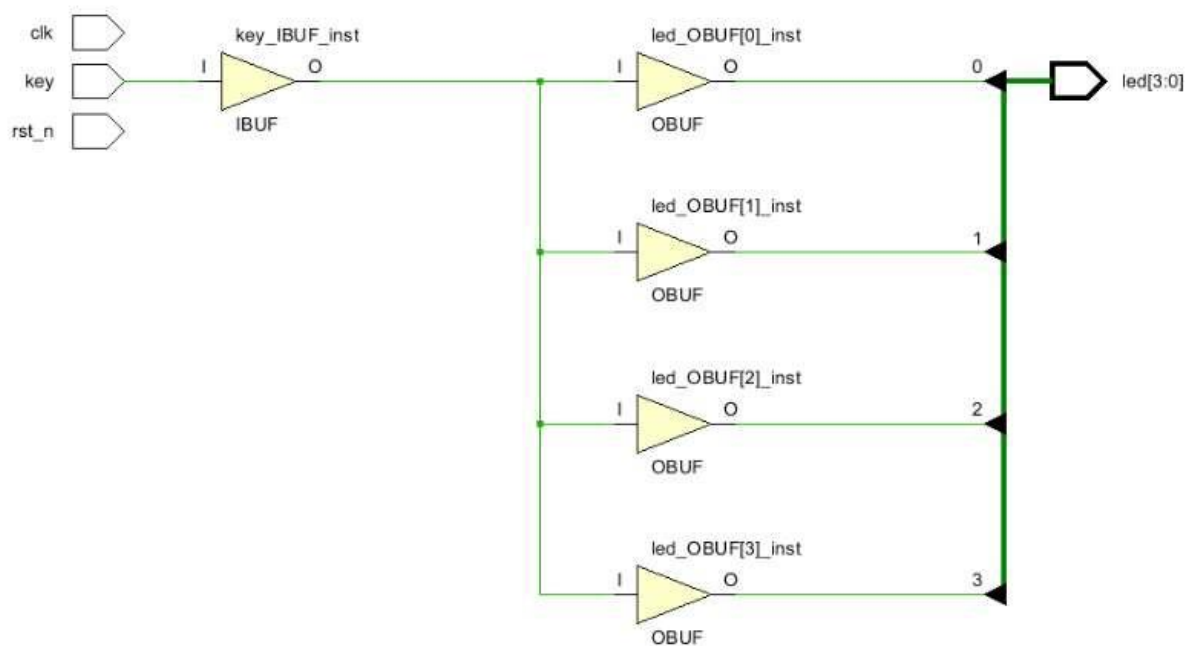
```

1  create_clock -period 20.000 -name clk -waveform {0.000 10.000}
2
3  set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
7  set_property PACKAGE_PIN J14 [get_ports {led[3]}]
8  set_property PACKAGE_PIN K14 [get_ports {led[2]}]
9  set_property PACKAGE_PIN J18 [get_ports {led[1]}]
10 set_property PACKAGE_PIN H18 [get_ports {led[0]}]
11 set_property PACKAGE_PIN M15 [get_ports key]
12 set_property PACKAGE_PIN L16 [get_ports rst_n]
13 set_property PACKAGE_PIN U18 [get_ports clk]
14 set_property IOSTANDARD LVCMOS33 [get_ports clk]
15 set_property IOSTANDARD LVCMOS33 [get_ports key]
16 set_property IOSTANDARD LVCMOS33 [get_ports rst_n]

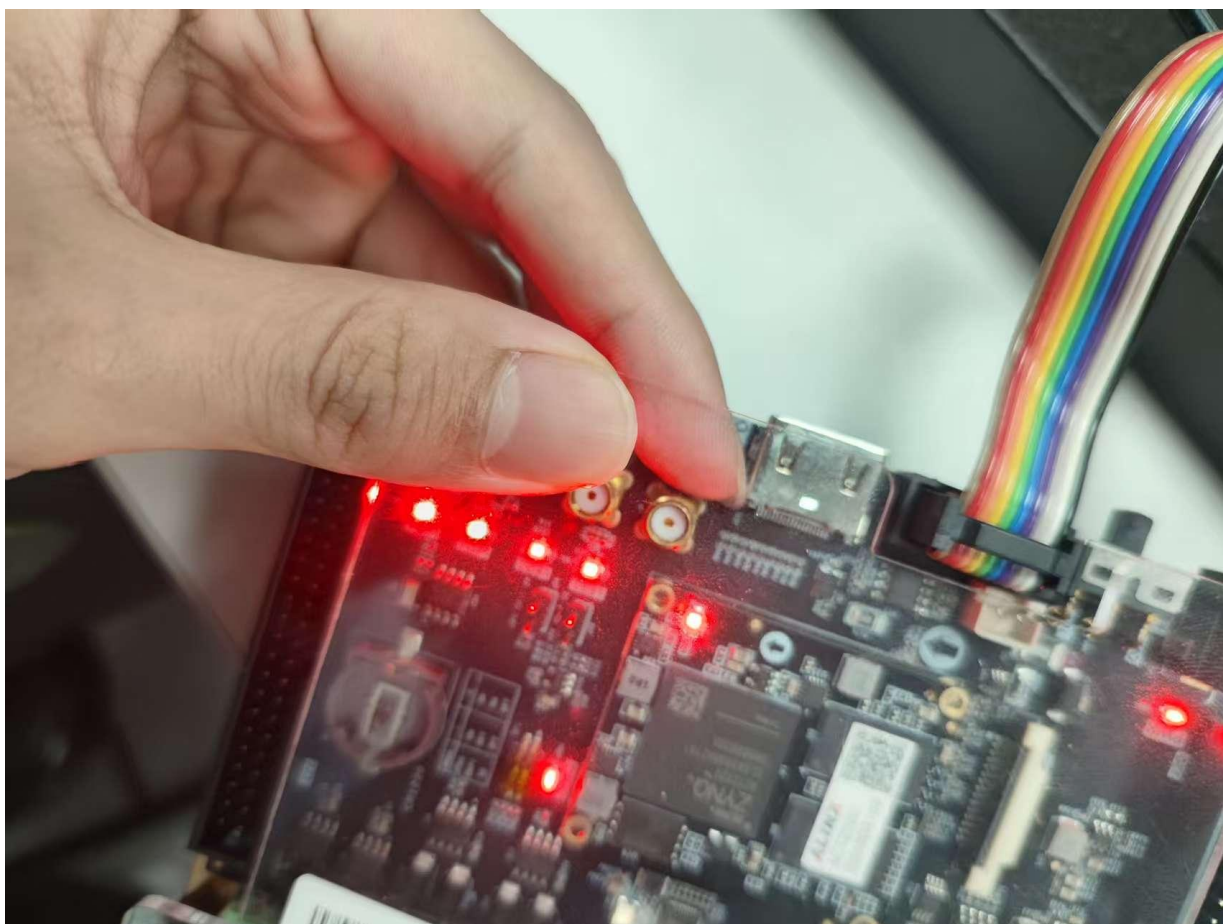
```

1.5 实验结果

1.5.1 结构图



1.5.2 FPGA 原型验证



1.6 结论

本实验成功基于 VIVADO 和 AX7Z020 开发板实现了按键控制 LED 灯的功能。通过查阅硬件手册完成了引脚分配，采用 Verilog 组合逻辑编写代码，经下载验证，按下按键时 LED 点亮，松开时熄灭，完全满足设计要求。实验掌握了 FPGA 开发的完整流程，熟悉了开发板资源的使用方法，达到了预期实验目的。

二 实验 2：基于计数器的循环灯设计

2.1 实验目的

- 1) 理解 FPGA 从 HDL 设计、验证设计、约束设计到板级测试的完整过程；
- 2) 理解 Verilog 模块结构、仿真激励结构；
- 3) 熟悉仿真工具 modelsim 的使用，testbench 的编写和时序的分析；

2.2 设计方案

2.2.1 系统功能概述

设计一个基于 FPGA 的计数器循环灯控制系统。系统输入 50MHz 板载时钟（clk）和复位信号（reset，高电平有效），输出 4 位 LED 信号（led[3:0]）。计数器每计数 6 个时钟周期，LED 输出状态取反一次，实现循环闪烁效果。本实验重点在于 testbench 仿真、时序分析。

2.2.2 硬件资源分配

计数器：采用内部寄存器实现，位宽为 3 位（最大可计数值为 7），本实验中计数范围为 0~5。每个时钟上升沿到来时，计数值加 1。当计数值达到 5 时，表示已计数满 6 次，此时计数值归零。

LED 输出：选择四个独立 LED（LED0~LED3），每个 LED 对应一个 FPGA 引脚，引脚分配分别为 J14, K14, J18, H18。LED 引脚为共阳极，向引脚提供低电平时 LED 点亮。

2.2.3 端口定义

端口名称	方向	位宽	说明
clk	input	1	时钟
rst_n	input	1	复位

led	output	4	LED 输出
-----	--------	---	--------

2.2.4 实验设备

- 1) 安装有 VIVADO 软件的 PC 机;
- 2) AX7Z020 开发板;

2.3 RTL 模块设计

```

22
23  module test
24  (
25      input clk,
26      input rst_n,
27
28      output [3:0] led
29  );
30
31      reg [3:0] count;
32      reg flag;
33      always @(posedge clk) begin
34          if(!rst_n) begin
35              count <= 32'h0;
36              flag <= 1;
37          end else if(count == 4'd5) begin
38              count <= 4'd0;
39              flag <= ~flag;
40          end else begin
41              count <= count + 1;
42          end
43      end
44
45      assign led = {4{flag}};
46
47  endmodule
48

```

模块具体实现功能如下:

- a) 当复位即 $\text{rst_n} == 0$ 时, 计数器 $\text{count} \leq 0$, led 标志位赋值为 1(熄灭)
- b) 置位并且时钟上升沿来临时, 若 $\text{count} == 5$, 则 $\text{count} \leq 0$, led 取反, 否则计数器加 1

2.4 约束脚本设计

根据 AX7Z020 原理图，查找按键和 LED 对应的 FPGA 管脚号，在 VIVADO 中创建 XDC 约束文件，如下图所示：

```
create_clock -period 20.000 -name clk -waveform {0.000 10.000}

set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
set_property PACKAGE_PIN J14 [get_ports {led[3]}]
set_property PACKAGE_PIN K14 [get_ports {led[2]}]
set_property PACKAGE_PIN J18 [get_ports {led[1]}]
set_property PACKAGE_PIN H18 [get_ports {led[0]}]
set_property PACKAGE_PIN L16 [get_ports rst_n]
set_property PACKAGE_PIN U18 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports rst_n]
```

2.5 仿真文件设计

2.5.1 仿真目标

- 1) 验证复位功能：reset 有效时计数器清零，LED 恢复初始状态
- 2) 验证计数功能：每 6 个时钟周期，LED 状态翻转一次
- 3) 验证连续多次计数翻转的正确性

2.5.2 测试向量设计

测试阶段	操作	预期结果
初始化	施加复位信号	count=0, flag = 1
复位释放	rst_n=1	开始计数
第 1~3 周期	计数递增	LED 不变
复位测试	施加复位信号	count=0, flag = 1

count==5	下一个周期	flag 信号翻转
-----------------	-------	-----------

count==5	下一个周期	flag 信号翻转
-----------------	-------	-----------

count==5	下一个周期	flag 信号翻转
-----------------	-------	-----------

2.5.3 testbench

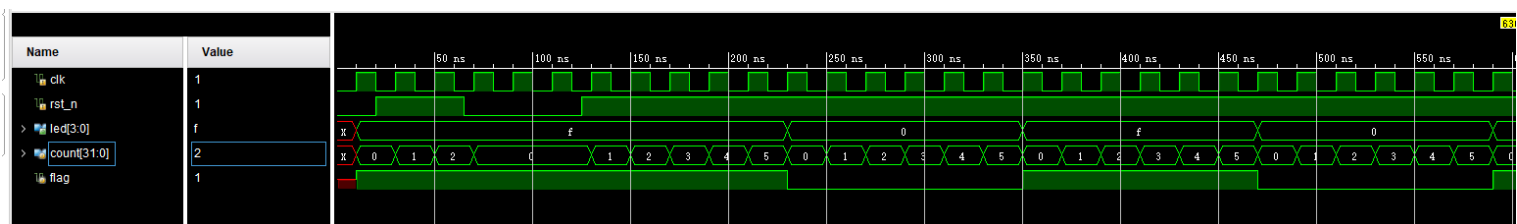
```

22
23 module tb;
24
25     reg clk;
26     reg rst_n;
27
28     wire [3:0] led;
29
30     initial begin
31         clk = 0;
32         rst_n = 0;
33         #20
34         rst_n = 1;
35         #45
36         rst_n = 0;
37         #60
38         rst_n = 1;
39     end
40
41     test u_test
42     (
43         .clk(clk),
44         .rst_n(rst_n),
45         .led(led)
46     );
47
48     always #10 clk = ~clk;
49
50 endmodule
51

```

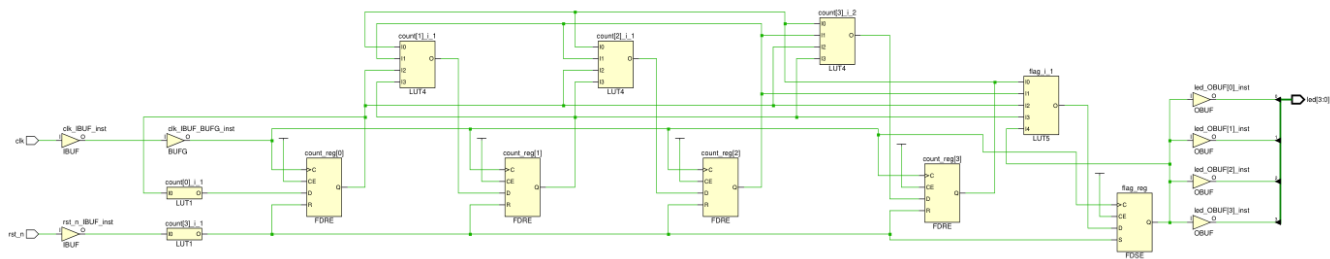
2.6 实验结果

2.6.1 仿真波形图



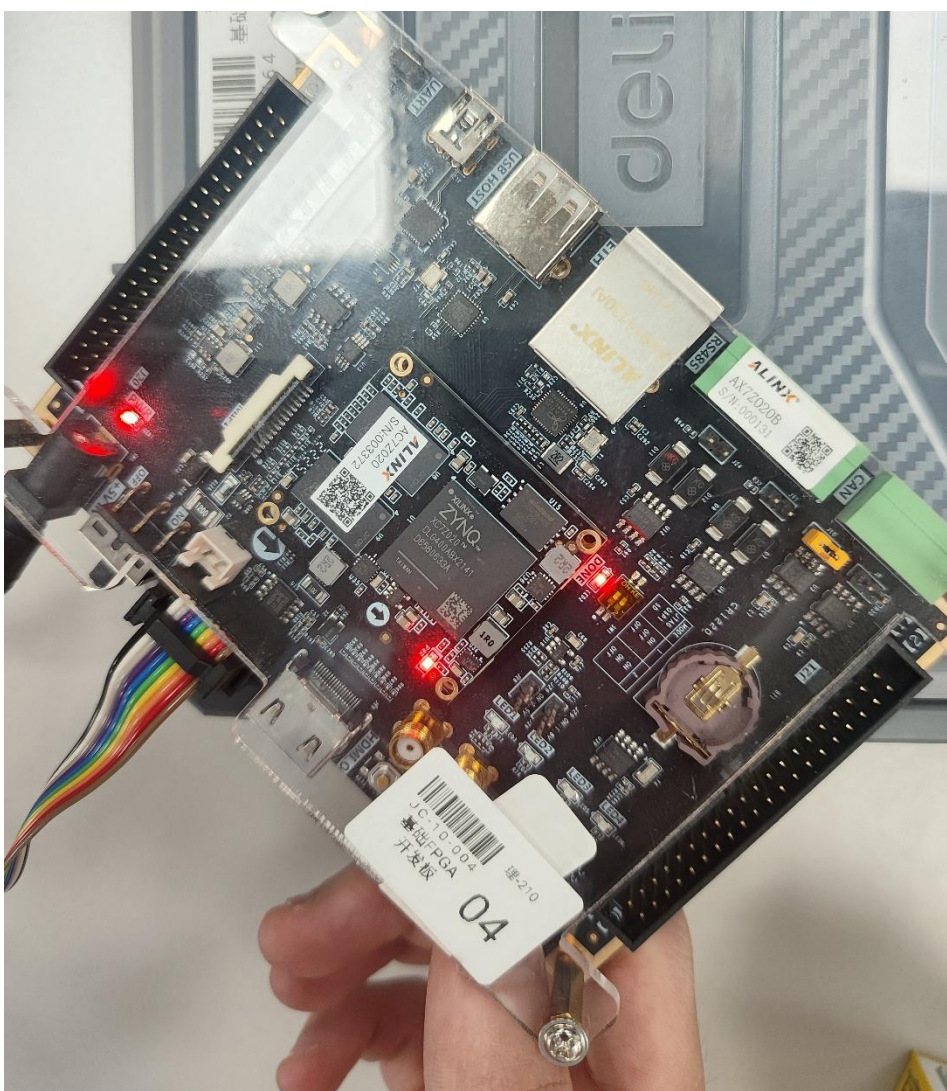
由波形图可知，设计满足仿真要求

2.6.2 结构图

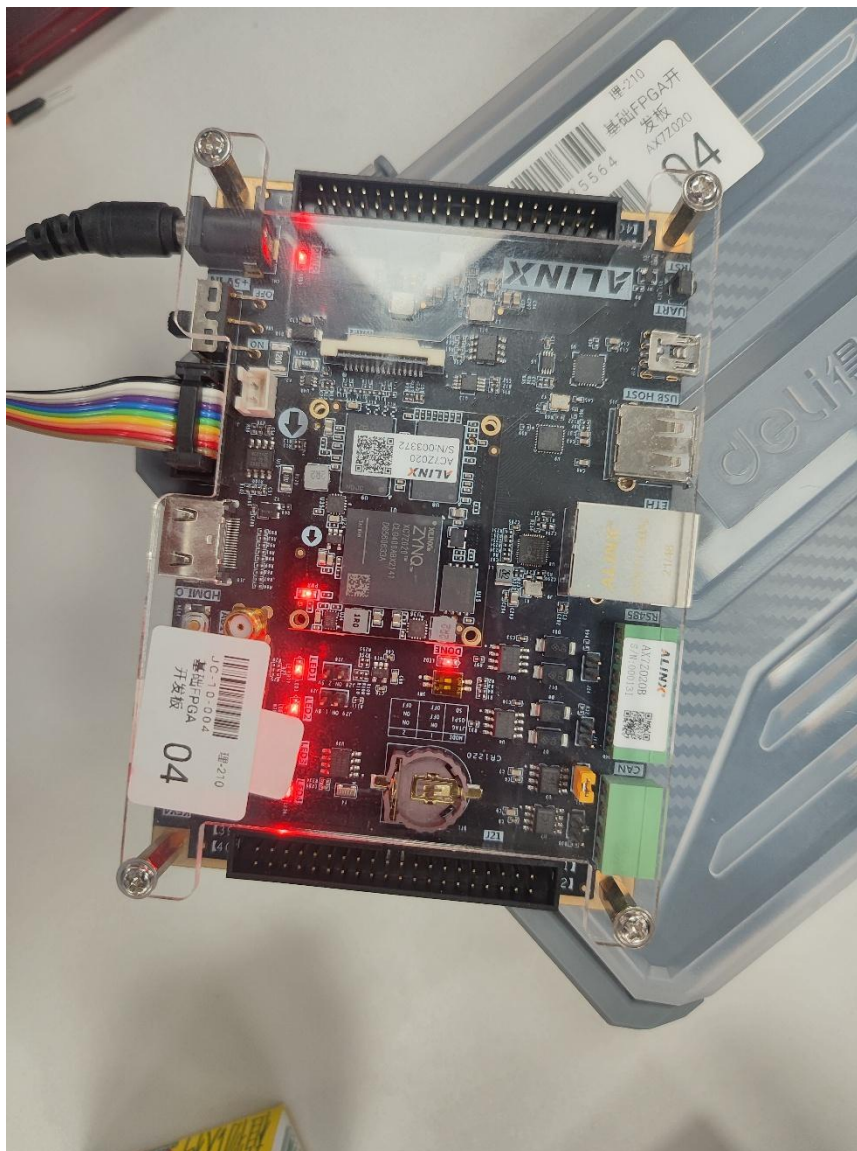


2.6.3 FPGA 原型验证

按键复位，LED 熄灭：



在 50M 时钟下，设置 LED 灯每 0.5 秒闪烁一次，即计数最大至 25000000，如下图所示：



2.7 结论

本实验成功基于 VIVADO 和 AX7Z020 开发板实现了基于计数器的循环灯控制功能。采用时序逻辑设计了一个 3 位计数器，每计数 6 个时钟周期 LED 状态取反一次，实现了 LED 循环闪烁效果。通过编写 testbench 进行了功能仿真，验证了复位功能、计数功能以及连续多次翻转的正确性。经下载验证，LED 在 50MHz 时钟下按预期每 0.5 秒闪烁一次，完全满足设计要求。本实验掌握了时序逻辑设计方法、testbench 编写技巧以及仿真分析流程，进一步加深了对 FPGA 开发全过程的理解。

三 实验 3：基于新思科技套件的数电设计、前仿和综合

3.1 实验目的

- 1) 理解专用集成电路的设计流程（从设计到综合）
- 2) 理解四位加法器的工作原理
- 3) 熟悉基于 Synopsys VCS 和 DC 的作用集成电路从设计、前仿和综合的过程和工具。

3.2 设计方案

3.2.1 系统功能概述

设计一个四位全加器（4-bit Full Adder），实现对两个四位二进制数（in1[3:0]和in2[3:0]）的加法运算，输出四位和信号（out[3:0]）以及一位进位输出（c）。本实验基于 Synopsys VCS 完成代码设计和前仿真，基于 Design Compiler（DC）完成逻辑综合，旨在熟悉专用集成电路（ASIC）的设计流程。

3.2.2 端口定义

端口名称	方向	位宽	说明
a	input	4	加数 1
b	input	4	加数 2
s	output	4	和输出
cout	output	1	进位输出

3.2.3 实验设备

安装有 VCS 和 DC 软件的 PC 机

3.3 RTL 模块设计

```
1
2  module full_adder
3  (
4      input [3:0] a,
5      input [3:0] b,
6
7      output [3:0] s,
8      output cout
9  );
10
11     assign {cout, s} = a + b;
12
13 endmodule
14
```

3.4 仿真文件设计

3.4.1 仿真目标

- 1) 验证基本加法功能：验证四位全加器在不同输入组合下的算术运算是否正确，即满足 $\{cout, s\} = a + b$
- 2) 验证边界值处理：测试最小输入（0+0）、最大输入（15+15）以及临界进位情况（7+8、8+8）
- 3) 验证随机输入正确性：通过随机数生成 20 组测试用例，验证各种组合下的加法正确性
- 4) 实现自动比对：通过 testbench 自动计算期望值并与实际输出比对，检测到错误时立即报告并终止仿真
- 5) 生成波形文件：通过 \$vcdpluson 生成波形文件，便于在 VCS 图形界面中观察输入输出信号时序

3.4.2 测试向量设计

测试阶段	操作	预期结果
------	----	------

最小值测试	a=0, b=0	s=0, cout=0
最大值测试	a=15, b=15	s=14, cout=1
边沿测试	a=7, b=8	s=15, cout=0
进位测试	a=8, b=8	s=0, cout=1
随机测试（20 组）	a,b 均为随机 0~15	s = (a+b)的低 4 位，cout = (a+b)的第 5 位

3.4.3 testbench

```

3 module tb;
4
5     reg [3:0] a, b;
6     wire [3:0] s;
7     wire cout;
8
9     reg [3:0] expect_s;
10    reg expect_cout;
11
12    full_adder dut
13    (
14        .a(a),
15        .b(b),
16        .s(s),
17        .cout(cout)
18    );
19
20    always @(*) begin
21        {expect_cout, expect_s} = a + b;
22
23        $display("a=%03d (%03h), b=%03d (%03h), expected={%03b,%03h}, actual={%03b,%03h}", a, a, b, b, expect_cout, expect_s, cout, s);
24
25        if (expect_s != s || expect_cout != cout) begin
26            $display("ERROR: Mismatch detected!");
27            $finish;
28        end
29    end
30 end

```

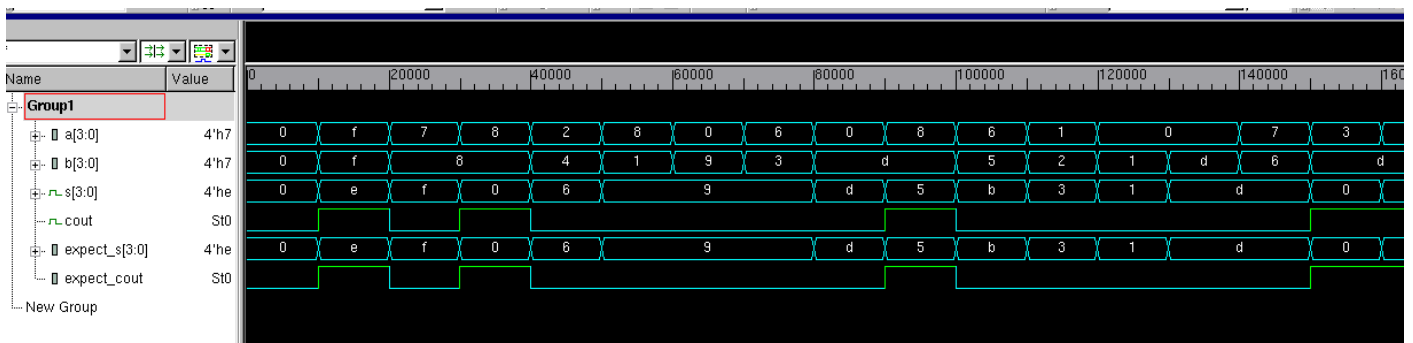
```

32     initial begin
33         $vcdpluson;
34
35         $display("The min bound test cases:");
36         a = 4'h0; b = 4'h0; #10;
37
38         $display("The max bound test cases:");
39         a = 4'hf; b = 4'hf; #10;
40
41         $display("The edge test cases:");
42         a = 4'h7; b = 4'h8; #10;
43
44         $display("The carry test cases:");
45         a = 4'h8; b = 4'h8; #10;
46
47         $display("The random test cases:");
48         repeat(20) begin
49             {a, b} = $random;
50             #10;
51         end
52
53         $finish;
54
55     end

```

3.5 VCS 仿真结果

3.5.1 仿真波形



3.5.2 log

```
5 VCD+ Writer I-2014.03_Full64 Copyright (c) 1991-2014
6 The min bound test cases:
7 a=00 (0), b=00 (0), expected={0,0}, actual={0,0}
8 The max bound test cases:
9 a=15 (f), b=15 (f), expected={1,e}, actual={1,e}
10 The edge test cases:
11 a=07 (7), b=08 (8), expected={0,f}, actual={0,f}
12 The carry test cases:
13 a=08 (8), b=08 (8), expected={1,0}, actual={1,0}
14 The random test cases:
15 a=02 (2), b=04 (4), expected={0,6}, actual={0,6}
16 a=08 (8), b=01 (1), expected={0,9}, actual={0,9}
17 a=00 (0), b=09 (9), expected={0,9}, actual={0,9}
18 a=06 (6), b=03 (3), expected={0,9}, actual={0,9}
19 a=00 (0), b=13 (d), expected={0,d}, actual={0,d}
20 a=08 (8), b=13 (d), expected={1,5}, actual={1,5}
21 a=06 (6), b=05 (5), expected={0,b}, actual={0,b}
22 a=01 (1), b=02 (2), expected={0,3}, actual={0,3}
23 a=00 (0), b=01 (1), expected={0,1}, actual={0,1}
24 a=00 (0), b=13 (d), expected={0,d}, actual={0,d}
25 a=07 (7), b=06 (6), expected={0,d}, actual={0,d}
26 a=03 (3), b=13 (d), expected={1,0}, actual={1,0}
27 a=14 (e), b=13 (d), expected={1,b}, actual={1,b}
28 a=08 (8), b=12 (c), expected={1,4}, actual={1,4}
29 a=15 (f), b=09 (9), expected={1,8}, actual={1,8}
30 a=12 (c), b=06 (6), expected={1,2}, actual={1,2}
31 a=12 (c), b=05 (5), expected={1,1}, actual={1,1}
32 a=10 (a), b=10 (a), expected={1,4}, actual={1,4}
33 a=14 (e), b=05 (5), expected={1,3}, actual={1,3}
34 a=07 (7), b=07 (7), expected={0,e}, actual={0,e}
35 $finish called from file "./rtl/tb.v", line 53.
36 $finish at simulation time                240000
```

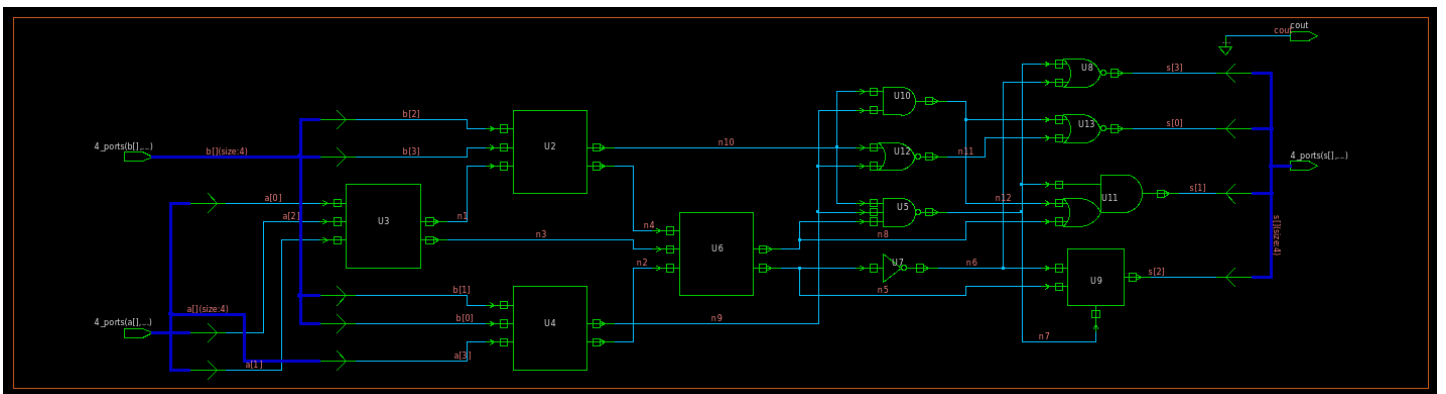
由上可知，边界情况以及随机情况都通过了测试

3.6 DC 综合结果

3.6.1 第一次综合

使用 compile 命令进行 DC 综合，得到网表以及结构图如下所示：

```
8 module full_adder ( a, b, s, cout );
9   input [3:0] a;
10  input [3:0] b;
11  output [3:0] s;
12  output cout;
13  wire n1, n2, n3, n4, n5, n6, n7, n8, n9, n10, n11, n12;
14  assign cout = 1'b0;
15
16  nd03d0 U5 ( .A1(n10), .A2(n9), .A3(n8), .ZN(n7) );
17  inv0d1 U7 ( .I(n5), .ZN(n6) );
18  nr02d0 U8 ( .A1(n7), .A2(n6), .ZN(s[3]) );
19  mx02d0 U9 ( .IO(n6), .I1(n5), .S(n7), .Z(s[2]) );
20  an02d1 U10 ( .A1(n10), .A2(n9), .Z(n12) );
21  ora21d1 U11 ( .B1(n12), .B2(n8), .A(n7), .Z(s[1]) );
22  nr02d0 U12 ( .A1(n10), .A2(n9), .ZN(n11) );
23  nr02d0 U13 ( .A1(n12), .A2(n11), .ZN(s[0]) );
24  ad01d0 U6 ( .A(n4), .B(n3), .CI(n2), .CO(n5), .S(n8) );
25  ad01d0 U4 ( .A(b[1]), .B(b[0]), .CI(a[3]), .CO(n2), .S(n9) );
26  ad01d0 U2 ( .A(b[2]), .B(b[3]), .CI(n1), .CO(n4), .S(n10) );
27  ad01d0 U3 ( .A(a[0]), .B(a[2]), .CI(a[1]), .CO(n3), .S(n1) );
28 endmodule
```



综合后，面积为 30.25：

```
l3
l4 Number of ports:                13
l5 Number of nets:                 25
l6 Number of cells:                13
l7 Number of combinational cells:  12
l8 Number of sequential cells:     1
l9 Number of macros/black boxes:   0
l0 Number of buf/inv:              1
l1 Number of references:            7
l2
l3 Combinational area:              30.250000
l4 Buf/Inv area:                   0.750000
l5 Noncombinational area:          0.000000
l6 Macro/Black Box area:           0.000000
l7 Net Interconnect area:          undefined (Wire load has zero net area)
l8
l9 Total cell area:                 30.250000
l0 Total area:                     undefined
l1 1
```

3.6.2 第二次综合

改进综合命令，使用 `compile_ultra`，得到以下结果：

```
7
8 module full_adder ( a, b, s, cout );
9   input [3:0] a;
10  input [3:0] b;
11  output [3:0] s;
12  output cout;
13  wire  n5, n6, n7, n8;
14
15  an02d1 U7 ( .A1(b[0]), .A2(a[0]), .Z(n6) );
16  nr02d0 U8 ( .A1(b[0]), .A2(a[0]), .ZN(n5) );
17  nr02d0 U9 ( .A1(n6), .A2(n5), .ZN(s[0]) );
18  ad01d0 U12 ( .A(b[3]), .B(a[3]), .CI(n8), .CO(cout), .S(s[3]) );
19  ad01d0 U11 ( .A(b[2]), .B(a[2]), .CI(n7), .CO(n8), .S(s[2]) );
20  ad01d0 U10 ( .A(b[1]), .B(a[1]), .CI(n6), .CO(n7), .S(s[1]) );
21 endmodule
```


四 总结

本次实验完成了三个数字集成电路设计项目，掌握了从 FPGA 到 ASIC 的完整设计流程：

实验 1 基于 VIVADO 实现了按键控制 LED 灯功能，熟悉了 FPGA 开发流程和 AX7Z020 开发板资源的使用。

实验 2 设计了基于计数器的循环灯控制系统，掌握了时序逻辑设计、testbench 编写及仿真分析。

实验 3 基于 Synopsys VCS 和 DC 工具完成了四位加法器的前仿和逻辑综合，采用 compile_ultra 优化后面积减少 40%。

通过本次课程实验，熟练掌握了 VIVADO、Modelsim、VCS、DC 等数字集成电路设计工具，建立了从 HDL 设计、仿真验证、约束设计、板级测试到逻辑综合的完整流程认知，为后续深入学习数字集成电路设计奠定了良好的实践基础。